

NLU Course Project - Part 1: NLU (LAB 5)

Stefano Camposilvan (257848)

University of Trento

stefano.camposilvan@studenti.unitn.it

1. Introduction

This report presents my contributions and results for Part 2 of the NLU course project at the University of Trento, focused on the joint task of intent classification and slot filling using the ATIS dataset. Intent classification consists in identifying the purpose of an input sentence, while slot filling assigns labels to relevant tokens (slots). The work is divided into two sections: Part A and Part B.

- **Part A** incrementally improves the proposed baseline LSTM model by adding bidirectionality and dropout layers.
- **Part B** involves loading a pre-trained BERT model and fine-tuning it for the joint task.

Across both parts, key hyperparameters, such as learning rate and batch size, were tuned to optimize performance, which were measured with F1-score (for slot filling) and accuracy (for intent classification).

2. Implementation Details

All modifications were applied incrementally, with performance evaluated at each step using loss curves and CSV-based logging. The total loss was computed as the sum of two independent cross-entropy losses, one for intent classification and one for slot filling. Each experiment was run for five times. The best-performing model was saved for testing, while the results reported in the apposite section are the average over the five runs. Early stopping with a patience of 3 epochs was employed. A test mode was also implemented to re-evaluate saved models and verify the logged results.

In **Part A**, the LSTM-based model was improved by introducing the following enhancements:

1. **Bidirectionality:** Enabled using PyTorch's LSTM `Bidirectional=True` flag. The output dimensions of subsequent layers were adjusted to account for the doubled hidden size.
2. **Dropout:** Applied after the LSTM layer and before both the slot filling and intent classification heads to mitigate overfitting.

In **Part B**, a pre-trained BERT model ("bert-base-uncased") was fine-tuned following the approach proposed by Chen et al. [1]. Specifically, the `[CLS]` token output was used for intent classification, while the token-level hidden states were used for slot filling.

The major challenge in applying the BERT model to this joint task was managing sub-token alignment due to BERT's use of WordPiece tokenization. This tokenizer, in fact, often splits words into multiple sub-tokens, making it difficult to align the original slot labels with the tokenized input. For instance, while a traditional tokenizer would keep "Burbank" as a single token, BERT splits it into: `['bur', '##bank']`, causing a mismatch in length between the input tokens and slot labels.

To address this, custom collation and loss masking logic were implemented from scratch to ensure precise sub-token handling. More specifically, each input sequence was tokenized using the tokenizer from `AutoTokenizer`, which allows to map tokens back to their original words through the `word_ids()` function. In fact, if a word is split into multiple tokens, all these tokens are assigned the same ID by this function, thus it was possible to assign the corresponding slot label only to the first sub-token of each word. All subsequent sub-tokens were instead assigned a special padding label (`PAD_TOKEN = 0`) to be ignored during loss computation, and additional padding was also applied at the beginning and end of the slot label sequence to account for BERT's special tokens: `[CLS]` and `[SEP]`. An example is proposed in Table 1.

For evaluation, slot label predictions and ground truth sequences were post-processed to remove entries corresponding to padding tokens, ensuring proper alignment before computing the F1 score, while intent classification was evaluated using standard accuracy, as done in part A.

3. Results

Extensive hyperparameter tuning was performed to optimize model performance. The final results and corresponding hyperparameter configurations for the best models are presented in the tables on the following page. A summary of key findings and conclusions for each part is provided here.

Part A: As shown in Table 2, both proposed modifications, bidirectionality and dropout, led to an improvement over the baseline. Specifically, enabling bidirectionality alone increased performance (F1 score of 0.941 ± 0.003 , and intent accuracy of 0.946 ± 0.003), while adding dropout layers to the best bidirectional configuration further improved results, achieving an F1 score of 0.948 ± 0.001 and an intent accuracy of 0.955 ± 0.001 .

In terms of hyperparameter values, models with larger hidden and embedding dimensions (e.g., 700/800) performed worse than those with smaller sizes (e.g., 200/300). Among the tested dropout probabilities (ranging from 0.1 to 0.5), a value of 0.3 consistently produced the best performance. Similarly, learning rates in the range of 1×10^{-5} to 5×10^{-5} proved most effective, while higher rates led to underfitting.

Part B: Fine-tuning the pre-trained BERT model produced better performance than the LSTM architecture used in Part A. Through experiments, I observed that small variations in learning rate, dropout probability and batch size had only a marginal impact on performance, thus indicating that BERT is not as susceptible to hyperparameter tuning as the LSTM used in part A is. The best configuration, using a dropout probability of 0.3, a learning rate of 5×10^{-5} , and a batch size of 128, achieved an F1 score of **0.954 ± 0.001** and an intent accuracy of **0.974 ± 0.001** , a significant improvement over the best LSTM configuration, assessing the superiority of transformer-based architectures for this class of sequence labeling tasks.

4. References

- [1] Q. Chen, Z. Zhuo, and W. Wang, "Bert for joint intent classification and slot filling," *arXiv preprint arXiv:1902.10909*, 2019. [Online]. Available: <https://arxiv.org/abs/1902.10909>

Table 1: *Example of slot label alignment for BERT WordPiece tokenization.*

Original sentence:

I would like a ticket from New York to
Burbank for Monday please

Tokens	Adjusted Labels
[CLS]	PAD_TOKEN
[i]	O
[would]	O
[like]	O
[a]	O
[ticket]	O
[from]	O
[new]	B-FromCity
[york]	I-FromCity
[to]	O
[bur]	B-ToCity
[##bank]	PAD_TOKEN
[for]	O
[monday]	B-Date
[please]	O
[SEP]	PAD_TOKEN

Table 2: *Part A: Incremental Model Improvements with Hyper-parameters and Test results of the best saved models. The best result is highlighted.*

Model Config	lr	Batch Size	Hid/Emb Size	Dropout	F1 Score	Intent Accuracy
Base LSTM	5×10^{-5}	128	200/300	–	0.922	0.934
+ Bidirectionality	5×10^{-5}	128	200/300	–	0.944	0.949
+ Dropout	5×10^{-5}	128	200/300	0.3	0.947	0.954

Table 3: *Part B: Test results of the best saved model.*

Model Config	lr	Batch Size	Hid/Emb Size	Dropout	F1 Score	Intent Accuracy
bert-base-uncased	5×10^{-5}	128	200/300	0.3	0.954	0.974